

VoltGrid — Architecture, Security & Judge Defense Master Guide

Target Audience: Technical review panels, hackathon evaluators, and engineering students preparing for project defense.

Platform: VoltGrid EV Mobility & Energy-Aware Routing Intelligence.

Table of Contents

1. System Overview & Execution Flow
2. Demystifying APIs & Open Source for Judges
3. The Judge's Security Question (30–60 Second Pitch)
4. Security Concepts Explained from Scratch
5. EV Battery Physics & Routing Algorithms
6. 30 Rapid-Fire Questions a Judge Might Ask
7. Prototype Honesty vs. Production Roadmap

1. System Overview & Execution Flow

1.1 The Simple Explanation (For Non-Software Judges)

Imagine you want to drive from Chennai to Madurai in an electric vehicle.

1. You select your vehicle (e.g. Tata Punch EV or Ather 450X), current battery percentage (e.g. 85%), and destination.
 2. The browser bundles your selections into an encrypted request (HTTPS) and sends it to our server.
 3. The server validates that the request is properly formatted and ensures it is not part of a denial-of-service attack.
 4. The server queries a road routing engine (OSRM) to calculate the actual highway turns and distances, rather than a straight line across a map.
 5. Our first-principles physics model computes the vehicle's true energy consumption based on passenger count, luggage weight, highway speeds, and road elevation.
 6. The engine checks if the vehicle can reach the destination on the current charge while preserving a safety buffer:
 - **Yes:** It returns a direct route with **0 charging stops**.
 - **No:** It searches our database of **925+ charging stations** along that specific highway corridor.
1. Candidate stations are ranked by:
 - Arrival battery margin (safe reachability)
 - Predicted availability at the vehicle's estimated arrival time (ETA)
 - Charging power (kW) and turnaround duration
 - Detour distance from the highway
 1. The server returns the optimized route, charging plan, battery projections, and alternatives.
 2. The frontend renders the route on an interactive map with turn-by-turn segments and charging cards.

1.2 The Deep Technical Pipeline (Under the Hood)



2. Demystifying APIs & Open Source for Judges

2.1 The Essential Taxonomy

TERM	EXACT DEFINITION	VOLTGRID IMPLEMENTATION
Open Source	Software whose source code is publicly accessible, modifiable, and inspectable under an open license.	Next.js, React, Tailwind CSS, Leaflet, Vitest, Zod, and the OSRM C++ engine.
Open Data	Data freely accessible, reusable, and redistributable without intellectual property restrictions.	OpenStreetMap (OSM) highway road network.
Public API	A programmatic network endpoint accessible over the internet for third-party client integration.	OSRM demo routing endpoint (router.project-osrm.org), Nominatim geocoder.
Free API	An API that charges zero monetary fees, though it may enforce rate limits or require an authentication token.	Open Charge Map API (api.openchargemap.io).
Self-Hosted Service	Software deployed, operated, and maintained on private servers or cloud instances (e.g. AWS EC2, Docker).	In production, VoltGrid would host its own containerized OSRM cluster.
Third-Party Service	An external server infrastructure owned and managed by an independent organization outside your boundary.	The public Overpass API instance (overpass-api.de).

2.2 External Services Used in VoltGrid

1. Open Source Routing Machine (OSRM)

- **Endpoint:** <https://router.project-osrm.org/route/v1/driving/{coords}?overview=full&geometries=geojson>
- **Function:** Turn-by-turn road network routing, highway segment distances, driving durations, and exact road geometry.
- **Why VoltGrid uses it:** Provides actual road curves and distances across Indian national highways instead of straight-line estimates.
- **Data Sent:** Only the coordinate pairs: `lng1,lat1;lng2,lat2`. Zero user identities or credentials are sent.
- **API Key:** None required.
- **Classification:** Public API hosting an open-source engine using open-data road maps.
- **Failure Resilience:** Protected by an `AbortSignal.timeout(10000)` (10-second timeout). Falls back to haversine distance if the external network is unreachable.

2. OpenStreetMap Nominatim

- **Endpoint:** <https://nominatim.openstreetmap.org/search?q={query}&format=json&countrycodes=in&viewbox={viewbox}>
- **Function:** Place name search and geocoding into coordinates (`latitude, longitude`).
- **Data Sent:** Search query string, bounded to Tamil Nadu, Puducherry, and Karnataka.
- **API Key:** None required.
- **Protection:** 300ms debounce timer prevents excessive network requests during typing.

3. Open Charge Map (OCM)

- **Endpoint:** <https://api.openchargemap.io/v3/poi/?countrycode=IN&latitude=...&longitude=...>
- **Function:** EV charging station registry (locations, operators, connector types, power ratings).
- **Data Sent:** Center coordinates, search radius, and server-side API key.
- **API Key:** Required (`OCM_API_KEY`).
- **Security & Caching:** The key is stored strictly on the server in `.env.local` and is never exposed to the client. Responses are cached locally (`.data/station-coverage.json`) with a 24-hour TTL.

4. OpenStreetMap Overpass API

- **Endpoint:** <https://overpass-api.de/api/interpreter>
- **Function:** Searches for verified lodging (`tourism=hotel` , `tourism=guest_house` , `tourism=motel`) within 8 km of battery exhaustion points.
- **Data Sent:** Read-only spatial bounding box query.
- **API Key:** None required.

- **Failure Resilience:** Wrapped in `try/catch` returning `[]` on failure; never interrupts route optimization.

3. The Judge's Security Question (30–60 Second Pitch)

Judge Asks: "Your APIs are open source and public. Doesn't that mean anyone can hack your system, tamper with routes, or steal your data?"

Memorable Defense Speech:

*"No, sir/ma'am. That reflects a common misconception regarding open source versus system security.

In cybersecurity, **Kerckhoffs's Principle** dictates that a system's security must depend on its architecture and secrets, not on keeping algorithms hidden. Open-source engines like Linux, OpenSSL, and OSRM are global standards precisely because their code is openly audited by thousands of engineers.

In VoltGrid, we separate public data from system control through defense-in-depth:

1. **Zero Secret Exposure:** Third-party API keys (like Open Charge Map) are kept strictly on our backend server. No keys exist in client bundles.
2. **Strict Boundary Validation:** Every incoming payload is validated using Zod schemas and capped at 128 KiB to prevent injection and buffer exhaustion.
3. **Abuse Protection:** In-memory IP rate limiting mitigates automated attacks and denial-of-service attempts.
4. **Zero-Trust External Consumption:** We treat all external APIs as untrusted. All requests enforce strict 10-second timeouts and offline fallbacks so external outages cannot crash our system.
5. **Privacy by Design:** We transmit only geographic coordinate pairs—never user identities, passwords, or personal credentials."

4. Security Concepts Explained from Scratch

1. HTTPS

- **What it is:** HTTP encrypted via Transport Layer Security (TLS).
- **Protection:** Prevents eavesdropping and Man-in-the-Middle (MitM) packet tampering on public networks.

2. API Key Protection & Environment Variables

- **What it is:** An API key is an authentication token granting access to a service.
- **Danger of Client Exposure:** Any code delivered to the browser (JS/HTML) can be inspected in DevTools. Hardcoded client keys can be extracted and abused.
- **Implementation:** Secrets are stored in `.env.local` without the `NEXT_PUBLIC_` prefix, ensuring they remain strictly on the Node.js server.

3. Zod Runtime Validation

- **Why Frontend Validation Alone Fails:** Attackers can bypass browser forms using Postman, `curl`, or custom scripts.
- **Implementation:** Zod enforces strict schemas on the backend, rejecting malformed types, unexpected fields, and out-of-bounds numbers with HTTP 400.

4. Rate Limiting

- **Protection:** Prevents automated bots from overwhelming server compute resources (Denial of Service).
- **Implementation:** `apiGuard()` tracks IP requests via an in-memory token bucket. Clients exceeding 120 requests/minute receive HTTP 429 with `Retry-After`.

5. CORS (Cross-Origin Resource Sharing)

- **Protection:** Restricts unauthorized external websites from executing requests against our backend APIs from a user's browser session.
- **Implementation:** Explicit whitelist (`NEXT_PUBLIC_APP_URL` and `localhost`). Wildcards (`*`) are disallowed.

6. CSP (Content Security Policy)

- **Protection:** Mitigates Cross-Site Scripting (XSS) and unauthorized data exfiltration.
- **Implementation:** Configured in `next.config.ts`, restricting script execution and network calls strictly to approved domains.

7. HSTS (HTTP Strict Transport Security)

- **Protection:** Forces browsers to establish secure HTTPS connections exclusively (`max-age=63072000; includeSubDomains; preload`), mitigating SSL-stripping.

8. SSRF (Server-Side Request Forgery)

- **Protection:** Prevents attackers from coercing the backend into dispatching requests to internal private infrastructure (e.g. cloud metadata services).
- **Implementation:** Target URLs for external fetches (OSRM, OCM, Overpass) have static, hardcoded hosts. User input is never concatenated into destination hostnames.

9. Supply Chain Security & `npm audit`

- **Protection:** Identifies known security vulnerabilities in third-party npm packages.
- **Implementation:** Automated CI pipeline (`.github/workflows/ci.yml`) executes `npm audit --audit-level=high` on every commit.

10. Payment Security & Cryptographic Signature Verification (HMAC-SHA256)

- **Protection:** Prevents client-side payment forgery, price tampering, and unpaid slot reservation exploits.
- **Implementation:** Secrets (`RAZORPAY_KEY_SECRET`) are isolated exclusively on the server. When client completes checkout, `/api/payments/razorpay/verify-payment` calculates the cryptographic HMAC-SHA256 digest over `${order_id}|${payment_id}` using Node.js `crypto` and verifies exact signature matching before confirming slot reservations.

11. Role-Based Access Control (RBAC) & PortalGuard

- **Protection:** Prevents unauthorized lateral privilege escalation between driver, operator, and community host views.
- **Implementation:** Evaluated on client and server boundaries using `PortalGuard` (`src/lib/auth/AuthContext.tsx`). Sessions synchronize between browser `localStorage` and HTTP cookies (`voltgrid_role`, `voltgrid_session`), redirecting unauthenticated or wrong-role users to their appropriate portal homepage.

5. EV Battery Physics & Routing Algorithms

5.1 First-Principles Energy Modeling

$$\text{Usable Energy (kWh)} = \text{Pack Capacity (kWh)} \times \frac{\text{SoC}_{\text{start}} - \text{SoC}_{\text{reserve}}}{100}$$

$$\text{Adjusted Demand (Wh/km)} = \text{Base Wh/km} \times M_{\text{payload}} \times M_{\text{terrain}} \times M_{\text{traffic}} \times M_{\text{weather}} + P_{\text{cargo}}$$

$$\text{Effective Range (km)} = (\text{Usable Energy (kWh)} \times 1000) / (\text{Adjusted Demand (Wh/km)})$$

Parameters:

- **Base Consumption:** Laboratory baseline (e.g. 36 Wh/km for Ather 450X, 146 Wh/km for Tata Nexon EV).

- **Payload Multipliers:**
- **2W:** Solo rider (1.00) vs. Pillion passenger (1.15) accounting for frontal wind area.
- **4W:** Driver (1.00) scaling +3.5% per additional occupant.
- **Cargo Penalty:** Additive rate (+0.04 Wh/km per kg of luggage).
- **Battery Chemistries:**
- **LFP (Lithium Iron Phosphate):** High thermal stability, safe for routine 100% charging.
- **NMC (Nickel Manganese Cobalt):** Higher energy density, recommended 85–90% operating ceiling to reduce wear.

5.2 2-Wheeler vs. 4-Wheeler Characteristics

DIMENSION	2-WHEELER (LIGHT EV)	4-WHEELER (PASSENGER EV)
Pack Capacity	2.0 – 5.1 kWh	24.0 – 82.6 kWh
Effective Range	45 – 140 km	180 – 500 km
Nominal Demand	30 – 45 Wh/km	120 – 165 Wh/km
Charging Interface	Standard 15A/16A AC Socket	CCS-2 DC Fast Charging / Type 2 AC
Highway Sensitivity	High aerodynamic drag / pillion mass	High cruising speeds (70–100 km/h) / cabin AC
Speed Range (Fastest)	50 – 65 km/h	70 – 90 km/h
Speed Range (Efficient)	40 – 50 km/h	55 – 70 km/h

6. 30 Rapid-Fire Questions a Judge Might Ask

Q1: What is VoltGrid?

A: An energy-aware EV route optimization platform that integrates battery physics, payload mass, and arrival-time charger availability to deliver reliable intercity journey plans.

Q2: What core problem does it solve?

A: It eliminates EV range anxiety by preventing drivers from being stranded by unfeasible routes or offline/occupied charging stations.

Q3: Which routing engine are you using?

A: The Open Source Routing Machine (OSRM) driving API, which provides pairwise highway geometries and accurate travel durations.

Q4: Why did you choose OSRM over Google Maps?

A: OSRM is open-source, lightweight, eliminates proprietary tracking tokens, provides direct coordinate geometry, and can be self-hosted on private infrastructure.

Q5: Is OSRM safe to use?

A: Yes. All traffic is encrypted over HTTPS, requests enforce a 10-second timeout, and only coordinate waypoints are transmitted without personal user identities.

Q6: If OSRM is public, can't anyone hack it?

A: No. OSRM is a read-only query engine. Our backend validates all returned geometries and gracefully falls back to haversine math if the service fails.

Q7: Where are your API keys stored?

A: Strictly on the server in `.env.local`. They lack the `NEXT_PUBLIC_` prefix, preventing Next.js from bundling them into client JavaScript.

Q8: Can a user inspect the browser and steal your Open Charge Map key?

A: No. All OCM calls are executed server-side. The browser only interacts with our internal `/api/stations` endpoint.

Q9: What happens if OSRM goes down during calculation?

A: `fetchOSRM` catches the timeout and falls back to straight-line haversine distance calculation, returning a valid plan without crashing.

Q10: What security measures have you implemented on your APIs?

A: Zod schema validation, in-memory IP rate limiting, a 128 KiB payload ceiling, explicit CORS whitelisting, and strict headers (CSP, HSTS, X-Frame-Options).

Q11: How do you validate user input?

A: Using Zod schemas on the backend that verify data types, enforce numerical boundaries (e.g. SoC 0–100%), and reject unexpected fields.

Q12: How do you prevent API abuse or Denial of Service?

A: We enforce token-bucket rate limiting that caps requests at 120/minute per IP, blocking abusive clients with HTTP 429.

Q13: What is CORS and why is it needed?

A: Cross-Origin Resource Sharing prevents unauthorized third-party websites from executing cross-origin API calls using a user's browser session.

Q14: What is Content Security Policy (CSP)?

A: A security header restricting the external domains permitted to run scripts, render styles, or establish network connections.

Q15: How does VoltGrid calculate EV range?

A: By dividing usable battery energy (kWh) by the real-world adjusted consumption rate (Wh/km), which accounts for passenger payload, speed, and elevation.

Q16: How does it decide whether charging is required?

A: It simulates the route from origin to destination. If the estimated arrival battery level is above the user's requested safety reserve, it marks the route as direct (0 stops).

Q17: How does multi-stop charging work?

A: The `findMultiStop` algorithm selects forward charging stops along the highway corridor, ensuring the vehicle makes continuous progress toward the destination and never loops backwards.

Q18: How does it handle 2-wheelers and 4-wheelers differently?

A: 2-wheelers have small packs (2–5 kWh), AC socket charging, and high sensitivity to pillion wind drag. 4-wheelers have large packs (24–82 kWh), CCS-2 DC fast charging, and higher cruising consumption.

Q19: How does passenger load affect energy consumption?

A: Each additional passenger adds vehicle mass. In 2W, a pillion rider increases consumption by +15% due to aerodynamics. In 4W, each passenger adds ~3.5% load, plus cargo weight penalties.

Q20: What is State of Charge (SoC)?

A: The percentage of electrical energy remaining in the battery pack relative to its rated capacity (0% = empty, 100% = full).

Q21: What is a charging-aware route?

A: A route where charging stops are mathematically integrated into the path based on battery feasibility, charger availability, and queue times, rather than added as an afterthought.

Q22: What happens if a charging station fails while a driver is en route?

A: The application detects the outage and triggers autonomous re-routing (`/api/reroute`), selecting the next optimal charger and updating ETA and battery margins instantly.

Q23: How do you demonstrate charger failures to an evaluator?

A: Through our Operator Sandbox (`DemoControls`), where clicking "Simulate Outage" takes a station offline in memory to show live re-routing.

Q24: How does VoltGrid handle long-distance trips without snapping issues?

A: It uses pairwise leg-by-leg routing (`Origin → Stop 1` , `Stop 1 → Stop 2` , etc.), preventing OSRM from snapping to unrelated parallel roads.

Q25: How does VoltGrid pick the best charging station?

A: Using a weighted multi-factor scoring algorithm that balances predicted availability, detour distance, charging duration, station reliability, and energy price.

Q26: What is the difference between claimed range and effective range?

A: Claimed range is derived in ideal laboratory test cycles. Effective range is the real-world distance calculated from usable battery capacity and speed/payload-adjusted Wh/km.

Q27: How many charging stations and vehicle models does VoltGrid support?

A: Over 925 charging stations across Tamil Nadu and Bengaluru, and 26 Indian EV models (14 two-wheelers and 12 four-wheelers).

Q28: How does VoltGrid help drivers if a trip is completely out of reach?

A: It queries OpenStreetMap's Overpass API to locate hotels and guest houses near the battery exhaustion point for overnight charging.

Q29: What automated testing do you have?

A: 54 automated unit tests running in Vitest covering the optimizer, battery physics, payload models, desired arrival targets, security boundaries, and emergency swapping navigation (100% passing).

Q30: What would be required to transition VoltGrid from a prototype to full production?

A: Self-hosting our own OSRM cluster, migrating rate limiting to Redis, integrating direct OCPI feeds from CPOs, and deploying behind an enterprise Web Application Firewall (WAF).

Q31: How do payments work and how do you prevent fake bookings or payment tampering?

A: Payments are processed via the Razorpay payment gateway SDK. To prevent tampering, the frontend never determines payment validity. After user checkout, the backend route `/api/payments/razorpay/verify-payment` computes an HMAC-SHA256 signature using the server-side secret over the order ID and payment ID. A digital booking QR pass is generated only after cryptographic verification succeeds.

Q32: How is role-based authentication implemented?

A: VoltGrid implements a tripartite RBAC architecture for Drivers, Operators, and Hosts using `PortalGuard` (`src/lib/auth/AuthContext.tsx`). The session state is maintained across browser `localStorage` and HTTP cookies (`voltgrid_role`, `voltgrid_session`), enforcing automatic redirection if an unauthenticated user or unauthorized role attempts to access a protected portal.

Q33: How does VoltGrid help stranded EV drivers with critical battery (<10%)?

A: Through our Emergency Assistance & EV Helpline module (`/helpline`, `src/components/trip/EmergencyAssistance.tsx`). It immediately scans a 2 km micro-radius for emergency 15A commercial sockets, lists verified flatbed towing networks, and enables one-tap SOS dispatch escalations directly to the operator console (`/operator/sos`).

Q34: What is the Battery Swapping network integration?

A: For light 2-wheelers, fast-charging takes 45–60 minutes. VoltGrid indexes battery swapping kiosks (e.g. Battery Smart, Sun Mobility) alongside standard chargers. The routing algorithm factors in battery swapping as a 2-minute turnaround stop instead of a 45-minute charge cycle.

Q35: What is the Community Host network and how are earnings calculated?

A: The Host Portal (`/host/register`, `/host/earnings`) allows homeowners, offices, and shop owners to monetize idle EV chargers or 15A outdoor sockets. Hosts set custom pricing per kWh and availability windows. Earnings are calculated based on grid kWh delivered minus host platform fees.

Q36: How does the VoltBot AI Assistant operate safely without leaking credentials?

A: VoltBot (`src/components/ai/VoltBotAssistant.tsx`) is a rule-guided, context-aware mobility copilot. It provides dynamic quick actions tailored to the user's role (Driver, Operator, Host). It interacts strictly with validated public client endpoints and never consumes or outputs server secret keys.

Q37: How do you handle database migration when moving from hackathon to production?

A: We maintain a fully defined production DDL schema in `db/schema.sql`. It utilizes PostgreSQL with the PostGIS spatial extension (`GEOGRAPHY(POINT, 4326)` and `GEOGRAPHY(LINESTRING, 4326)`), indexed with spatial GIST indexes for sub-millisecond bounding box queries across millions of chargers.

Q38: What is the root cause problem VoltGrid solves that Google Maps cannot?

A: Google Maps routes for combustion engines using distance and traffic velocity. EV range is dynamic and non-linear—it changes based on passenger drag, cargo mass, and temperature. Furthermore, charger availability is dynamic; a charger showing green when leaving Chennai may be offline or occupied by an 8-car queue 90 minutes later. VoltGrid solves the "Blind ETA Crisis" by predicting charger vacancy at arrival time using a logistic regression model.

Q39: What happens if an external third-party API (OSRM or Nominatim) suffers an outage?

A: VoltGrid implements zero-trust external consumption. External requests are wrapped in strict 10-second `AbortSignal.timeout` controllers. If OSRM fails, the system automatically falls back to straight-line Haversine routing. If Nominatim fails, it serves from our local cache of 19 regional transportation hubs.

Q40: How does VoltGrid calculate Route Confidence Score (0–100%)?

A: Through `src/lib/models/confidence.ts`, which evaluates three independent dimensions: (1) Battery safety margin above manufacturer reserve, (2) Highway corridor charger density (backup chargers within reach), and (3) Real-time operator historical reliability and uptime.

7. Prototype Honesty vs. Production Roadmap

When speaking to evaluators, **never claim prototype features are enterprise production deployments**. Honesty demonstrates technical maturity:

CAPABILITY	CURRENT VOLTGRID IMPLEMENTATION	PRODUCTION REQUIREMENT
Road Routing	Public OSRM demo server with 10s timeout & haversine fallback	Dedicated self-hosted OSRM or Valhalla cluster with SLA
Rate Limiting	In-memory token bucket per Node.js process (120 req/min)	Distributed Redis cache across multi-region server clusters
CPO Telemetry	Hybrid model: 883 real OCM POIs + 42 high-density simulated corridor hubs	Live OCPI (Open Charge Point Interface) v2.2.1 feeds from Tata Power, Statiq, Zeon
Data Storage	In-memory simulation store with local disk cache (<code>.data/</code>) & client storage	PostgreSQL with PostGIS extension for spatial SQL queries (<code>db/schema.sql</code>)
Authentication & RBAC	<code>PortalGuard</code> with synchronized localStorage & HTTP cookies	Enterprise OAuth 2.0 / OIDC with JWT access tokens and MFA
Payments	Razorpay Node SDK with HMAC-SHA256 signature verification + dev test fallback	Live Razorpay / UPI gateway credentials with webhook idempotency
Elevation Data	Calibrated highway grade profiles	NASA SRTM 30m digital elevation model (DEM)
Unit Testing	54 Vitest automated tests covering physics, optimizer, security, & SOS	Full CI/CD integration with 85%+ branch coverage and E2E Cypress/Playwright